

Day 28–30: Collections Framework

Learn:

- List
- ArrayList
- LinkedList
- Vector

Practice:

- Student management using ArrayList

. First: What is a Collection?

Before understanding List, ArrayList, etc., first understand the problem they solve.

Suppose you want to store student marks.

You could create an array:

```
int[] marks = {80, 90, 75, 88, 95};
```

This works.

But imagine:

- Initially 5 students unnaru
- Later 3 students join అయ్యారు
- Now total 8 students
- Later 2 students leave అయ్యారు
- You need to add/remove/search students frequently

Normal arrays have a major limitation:

Array size is fixed once created.

Example:

```
int[] marks = new int[5];
```

This array can hold only 5 integers.

You cannot simply make it:

5 → 10

after creating it.

So Java provides a better way to handle groups of objects:

Collections Framework

2. What is the Java Collections Framework?

Java Collections Framework (JCF) is a set of **interfaces and classes** used to store and manipulate groups of objects.

Simple ga:

Multiple objects ni

store

add

remove

search

update

iterate

manage

cheyyadaniki Java provide chesina system = **Collections Framework**.

Examples:

List

Set

Queue

Map

Today మన focus:

List

└─ ArrayList

└─ LinkedList

└─ Vector

3. Why do we need Collections?

Let's compare an array and collection.

Array

```
String[] students = new String[5];
```

Capacity fixed.

ArrayList

```
ArrayList<String> students = new ArrayList<>();
```

Size can dynamically grow.

Suppose initially:

ArrayList

[Ram]

[Sita]

[Ravi]

Later:

[Ram]

[Sita]

[Ravi]

[John]

[Anu]

ArrayList automatically manages its internal storage as it grows.

So one major advantage is:

Collections can dynamically manage the amount of data.

4. Collection vs Collections

These two names confuse beginners.

Collection

Collection is an **interface**.

Collections

Collections is a **utility class** containing useful methods for collections.

For example:

```
Collections.sort(list);
```

```
Collections.reverse(list);
```

So:

Collection



Interface

Collections



Utility class

Don't mix them.

5. What is an Interface?

You already learned interfaces in OOP, but let's quickly connect it.

An interface tells us **what operations should be available**, without necessarily providing the complete implementation.

For example:

```
List<String> students;
```

Here:

List



Interface

It describes list operations such as:

```
add()
```

```
remove()
```

```
get()
```

```
set()
```

```
contains()
```

size()

But List itself isn't normally the object you instantiate.

You use an implementation such as:

ArrayList

LinkedList

Vector

6. What is List?

List is an **interface** in Java.

It represents an **ordered collection of elements**.

The important characteristics of a List are:

1. Ordered

Elements maintain their position/order.

Example:

Apple

Banana

Mango

Orange

The order is maintained.

2. Allows duplicates

You can have:

Apple

Apple

Mango

Apple

No problem.

3. Allows index-based access

Like arrays, elements have indexes.

Index: 0 1 2

Apple Mango Orange

So:

`list.get(0);`

returns:

Apple

7. List hierarchy

The important structure for today is:

Iterable

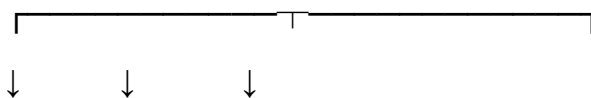
↓

Collection

↓

List

↓



ArrayList LinkedList Vector

Actually, LinkedList also implements Deque, but for our current learning, remember the above structure.

8. How to create a List?

There are two common ways.

Way 1

```
ArrayList<String> names = new ArrayList<>();
```

Way 2 — recommended in many situations

```
List<String> names = new ArrayList<>();
```

Why?

Because:

`List<String>`

is the interface, while:

`ArrayList<String>`

is the implementation.

This follows the idea:

Program to the interface, not the implementation.

For now, don't worry if that sentence feels advanced.

Just remember:

`List<String> names = new ArrayList<>();`

is very common Java code.

9. What is `<String>`?

This is extremely important.

`ArrayList<String>`

The `<String>` is called a **generic type parameter**.

It tells Java:

"This ArrayList should store Strings."

For example:

`ArrayList<String> names = new ArrayList<>();`

Allowed:

`names.add("Ravi");`

`names.add("Sita");`

`names.add("John");`

Not allowed:

`names.add(100);`

because 100 is an integer, not a String.

10. Why Generics?

Without generics, Java collections could store different types of objects.

For example:

"Ravi"

100

25.5

true

This can create problems.

Generics provide **type safety**.

```
ArrayList<String> names = new ArrayList<>();
```

means:

Only String values

Similarly:

```
ArrayList<Integer> marks = new ArrayList<>();
```

means:

Only Integer values

11. Important: Why Integer instead of int?

You cannot write:

```
ArrayList<int> marks;
```

✗ Wrong.

Instead:

```
ArrayList<Integer> marks;
```

Why?

Collections work with **objects**, not primitive types.

Java provides wrapper classes:

Primitive Wrapper

int	Integer
-----	---------

Primitive Wrapper

double Double

float Float

char Character

boolean Boolean

long Long

short Short

byte Byte

So:

`ArrayList<Integer>`

instead of:

`ArrayList<int>`

12. What is ArrayList?

ArrayList is a class that implements the List interface.

Its main purpose:

To store a dynamically growing list of objects.

Example:

```
ArrayList<String> names = new ArrayList<>();
```

Initially:

```
[]
```

Then:

```
names.add("Ravi");
```

Now:

```
[Ravi]
```

Then:

```
names.add("Sita");
```

Now:

[Ravi, Sita]

Then:

```
names.add("John");
```

Now:

[Ravi, Sita, John]

13. Importing ArrayList

You need:

```
import java.util.ArrayList;
```

Or:

```
import java.util.*;
```

ArrayList belongs to:

java.util

package.

So a basic program:

```
import java.util.ArrayList;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<String> names = new ArrayList<>();
```

```
        names.add("Ravi");
```

```
        names.add("Sita");
```

```
        names.add("John");
```

```
        System.out.println(names);
```

```
    }
```

```
}
```

Output:

[Ravi, Sita, John]

14. ArrayList internally — important concept

You may wonder:

"If an ArrayList is dynamic, how does Java actually store it?"

Internally, ArrayList uses an **array-like internal storage**.

Conceptually:

ArrayList

↓

Internal array

↓

[Ravi][Sita][John][][][]

When the current internal capacity becomes insufficient, ArrayList creates a larger internal array and moves/copies the elements.

So:

Old storage

[10][20][30][40]

↓ capacity insufficient

New larger storage

[10][20][30][40][][][]

This is why ArrayList is called **dynamic**.

You don't manually resize it.

15. ArrayList methods

These are the most important methods you should know.

add()
get()
set()
remove()
size()
contains()
isEmpty()
clear()
indexOf()

Let's learn each one.

16. add()

Used to add an element.

```
ArrayList<String> names = new ArrayList<>();
```

```
names.add("Ravi");  
names.add("Sita");  
names.add("John");
```

Result:

[Ravi, Sita, John]

17. Add at a specific index

You can specify the position.

```
names.add(1, "Anu");
```

Before:

Index

0 1 2

Ravi Sita John

After:

Index

0 1 2 3

Ravi Anu Sita John

Important:

Existing elements shift right.

18. `get()`

Used to retrieve an element.

```
String name = names.get(0);
```

Example:

```
ArrayList<String> names = new ArrayList<>();
```

```
names.add("Ravi");
```

```
names.add("Sita");
```

```
names.add("John");
```

```
System.out.println(names.get(1));
```

Output:

Sita

Because:

0 → Ravi

1 → Sita

2 → John

19. `set()`

Used to **replace/update** an existing element.

```
names.set(1, "Anu");
```

Before:

[Ravi, Sita, John]

After:

[Ravi, Anu, John]

Remember:

add() → adds new element

set() → replaces existing element

20. remove()

Used to remove an element.

By index:

```
names.remove(1);
```

If:

[Ravi, Sita, John]

then:

[Ravi, John]

because index 1 was removed.

21. Very important remove() confusion

Consider:

```
ArrayList<Integer> numbers = new ArrayList<>();
```

```
numbers.add(10);
```

```
numbers.add(20);
```

```
numbers.add(30);
```

Now:

```
numbers.remove(1);
```

means:

Remove element at index 1.

So:

[10, 20, 30]

↑

index 1

Result:

[10, 30]

But if you want to remove the value 20, you can write:

```
numbers.remove(Integer.valueOf(20));
```

Result:

[10, 30]

This distinction is important in interviews.

22. size()

Returns number of elements.

```
System.out.println(names.size());
```

Example:

[Ravi, Sita, John]

Output:

3

Remember:

Array

array.length

ArrayList

list.size()

23. contains()

Checks whether an element exists.

```
names.contains("Ravi");
```

Returns:

true

If:

```
names.contains("Kiran");
```

returns:

false

24. isEmpty()

Checks whether the list has zero elements.

```
names.isEmpty();
```

Empty list:

```
[]
```

returns:

true

25. clear()

Removes all elements.

```
names.clear();
```

Before:

```
[Ravi, Sita, John]
```

After:

```
[]
```

26. indexOf()

Finds the index of an element.

```
names.indexOf("Sita");
```

If:

```
[Ravi, Sita, John]
```

result:

1

If the element doesn't exist, it returns:

27. Complete ArrayList example

```
import java.util.ArrayList;

public class Main {

    public static void main(String[] args) {

        ArrayList<String> names = new ArrayList<>();

        names.add("Ravi");
        names.add("Sita");
        names.add("John");

        System.out.println(names);

        System.out.println(names.get(1));

        names.set(1, "Anu");

        System.out.println(names);

        names.remove(0);

        System.out.println(names);

        System.out.println(names.size());
```

```
        System.out.println(names.contains("John"));
    }
}
```

Let's trace it.

Step 1

```
ArrayList<String> names = new ArrayList<>();
```

```
[]
```

Step 2

```
names.add("Ravi");
```

```
[Ravi]
```

Step 3

```
names.add("Sita");
```

```
[Ravi, Sita]
```

Step 4

```
names.add("John");
```

```
[Ravi, Sita, John]
```

Step 5

```
names.get(1)
```

Result:

Sita

Step 6

```
names.set(1, "Anu");
```

Now:

```
[Ravi, Anu, John]
```

Step 7

```
names.remove(0);
```

Now:

```
[Anu, John]
```

28. How to traverse an ArrayList?

You need to know how to print/process every element.

Method 1: Normal for loop

```
for (int i = 0; i < names.size(); i++) {  
    System.out.println(names.get(i));  
}
```

Output:

Ravi

Sita

John

29. Enhanced for loop

You can also use:

```
for (String name : names) {  
    System.out.println(name);  
}
```

Meaning:

For every String element inside names, temporarily store it in name.

Conceptually:

names

↓

Ravi → name

Sita → name

John → name

This is very commonly used.

30. Can ArrayList contain duplicate values?

Yes.

```
ArrayList<String> names = new ArrayList<>();
```

```
names.add("Ravi");
```

```
names.add("Ravi");
```

```
names.add("Sita");
```

Result:

```
[Ravi, Ravi, Sita]
```

Because List allows duplicates.

31. Can ArrayList contain null?

Yes.

```
names.add(null);
```

Example:

```
[Ravi, null, Sita]
```

ArrayList permits null elements.

32. ArrayList with custom objects

This is extremely important for real projects.

You don't only store:

String

Integer

You can store your own objects.

Suppose:

```
class Student {
```

```
    int id;
```

```
    String name;
```

```
    int marks;
```

```
    Student(int id, String name, int marks) {
```

```

        this.id = id;

        this.name = name;

        this.marks = marks;
    }
}

```

Now:

```
ArrayList<Student> students = new ArrayList<>();
```

You can store:

```
students.add(new Student(101, "Ravi", 85));
```

```
students.add(new Student(102, "Sita", 90));
```

```
students.add(new Student(103, "John", 78));
```

This is the foundation of our Student Management System later.

33. Now: What is LinkedList?

LinkedList is another implementation of the List interface.

```
LinkedList<String> names = new LinkedList<>();
```

It stores elements using a **linked structure of nodes**.

Conceptually:

[Ravi] → [Sita] → [John] → null

Each node contains:

Data

+

Link/reference to another node

Conceptually:

Node

```

+-----+-----+
| data  | next  |
+-----+-----+

```

34. ArrayList vs LinkedList — basic idea

ArrayList

Conceptually:

[Ravi][Sita][John][Anu]

Elements are stored in an array-backed structure.

LinkedList

Conceptually:

[Ravi] → [Sita] → [John] → [Anu]

Elements are connected through links.

This difference affects performance.

35. Why use LinkedList?

LinkedList can be useful when your application frequently performs insertions/removals at certain positions where linked-node operations are advantageous.

For example:

A → B → C → D

Suppose you remove C.

The links can be adjusted:

A → B → D

Whereas an array-backed list may need to shift subsequent elements.

But there is an important detail:

LinkedList is not automatically faster for every insertion/removal.

Because LinkedList may first need to **find the position**, and finding an arbitrary index takes time.

So don't memorize:

"LinkedList is always faster."

That statement is wrong.

36. LinkedList methods

Many methods are the same as ArrayList:

`add()`

`get()`

`set()`

`remove()`

`size()`

`contains()`

But LinkedList also provides methods useful for the beginning/end:

`addFirst()`

`addLast()`

`getFirst()`

`getLast()`

`removeFirst()`

`removeLast()`

Example:

```
LinkedList<String> names = new LinkedList<>();
```

```
names.add("B");
```

```
names.add("C");
```

```
names.addFirst("A");
```

```
names.addLast("D");
```

Result:

```
[A, B, C, D]
```

37. addFirst()

```
names.addFirst("A");
```

Before:

[B, C]

After:

[A, B, C]

38. addLast()

names.addLast("D");

Before:

[A, B, C]

After:

[A, B, C, D]

39. getFirst()

names.getFirst();

Returns the first element.

[A, B, C, D]

↑

Result:

A

40. getLast()

names.getLast();

Returns:

D

41. removeFirst()

names.removeFirst();

Before:

[A, B, C, D]

After:

[B, C, D]

42. removeLast()

```
names.removeLast();
```

Before:

[B, C, D]

After:

[B, C]

43. What is Vector?

Vector is another class that implements the List interface.

```
Vector<String> names = new Vector<>();
```

It is also a **dynamically growing array-based list**.

So conceptually:

Vector

↓

Dynamic array

44. Why is Vector different from ArrayList?

The major historical difference is **synchronization**.

Many of Vector's methods are synchronized.

Meaning, roughly:

Vector provides built-in synchronization for its operations, which can make certain concurrent uses safer but can also add synchronization overhead.

ArrayList is generally not synchronized by default.

For modern Java applications, ArrayList is usually the normal choice when you don't specifically need Vector's legacy behavior.

45. Vector example

```
import java.util.Vector;
```

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Vector<String> names = new Vector<>();  
  
        names.add("Ravi");  
        names.add("Sita");  
        names.add("John");  
  
        System.out.println(names);  
    }  
}
```

Output:

[Ravi, Sita, John]

46. Vector capacity

Vector has a concept of **capacity**.

For example:

```
Vector<Integer> numbers = new Vector<>();
```

It manages internal storage and can grow when needed.

You can inspect:

```
numbers.size();
```

and:

```
numbers.capacity();
```

Remember:

size = number of actual elements

capacity = amount of internal storage available

These are NOT the same thing.

47. Size vs Capacity

Suppose:

Vector:

Capacity = 10

Elements = 3

Then:

size() → 3

capacity() → 10

Conceptually:

[10][20][30][][][][][][]

↑

3 actual elements

10 storage positions

48. ArrayList vs Vector

Both are array-backed dynamic lists.

ArrayList

Dynamic

Not synchronized by default

Generally preferred for normal single-threaded list use

Vector

Dynamic

Legacy class

Methods are synchronized

Usually not the first choice for new code

For interview preparation, remember:

ArrayList is generally preferred over Vector unless a specific legacy/concurrency requirement calls for Vector.

49. ArrayList vs LinkedList vs Vector

Here is the important comparison.

Feature	ArrayList	LinkedList	Vector
Type	Class	Class	Class
Implements List	Yes	Yes	Yes
Internal structure	Dynamic array	Linked nodes	Dynamic array
Maintains order	Yes	Yes	Yes
Allows duplicates	Yes	Yes	Yes
Allows null	Yes	Yes	Yes
Random access	Fast	Slower	Fast
Synchronized by default	No	No	Yes
Legacy class	No	No	Yes
Common general-purpose choice	Yes	Depends on use case	Less common

50. Time Complexity — important for interviews

You don't need to memorize everything immediately, but understand the idea.

ArrayList

get(index) → $O(1)$

set(index) → $O(1)$

add(end) → amortized $O(1)$

search → $O(n)$

insert middle → $O(n)$

remove middle → $O(n)$

Why middle insertion/removal is $O(n)$?

Because elements may need to shift.

Example:

[A][B][C][D][E]

Insert X at index 2:

[A][B][X][C][D][E]

C, D, E may need to shift.

51. LinkedList complexity

Typical:

get(index) $\rightarrow O(n)$

search $\rightarrow O(n)$

insert/remove at known node position $\rightarrow O(1)$

insert/remove by index $\rightarrow O(n)$ to locate position

The key phrase is:

If you already have the node/position, link changes can be $O(1)$; finding an arbitrary index can take $O(n)$.

52. Vector complexity

Similar array-backed behavior:

get(index) $\rightarrow O(1)$

set(index) $\rightarrow O(1)$

search $\rightarrow O(n)$

middle insertion $\rightarrow O(n)$

middle removal $\rightarrow O(n)$

Synchronization can add overhead compared with ArrayList.

53. When should you use ArrayList?

Use ArrayList when:

- You frequently access elements by index.
- You mostly add elements to the end.
- You need a general-purpose List.
- You don't need Vector's legacy synchronization behavior.

Example:

Student records

Product catalog

Employee list

Marks list

User list

54. When should you use LinkedList?

Use LinkedList when its linked structure and deque operations fit your workload.

It is particularly useful when you need operations at both ends:

`addFirst()`

`addLast()`

`removeFirst()`

`removeLast()`

For queue/deque-style behavior, ArrayDeque is often a better modern choice, but we'll study that when we reach Queue/Deque.

55. When should you use Vector?

Mostly:

Legacy applications

Existing code that specifically depends on Vector

For most new code:

ArrayList

is the usual choice for a general-purpose List.

56. Important: List itself cannot normally be instantiated

This is wrong:

```
List<String> names = new List<>();
```

✗ You cannot instantiate an interface.

Instead:

```
List<String> names = new ArrayList<>();
```

or:

```
List<String> names = new LinkedList<>();
```

This is because:

List

↓

Interface

ArrayList

↓

Class

LinkedList

↓

Class

57. Why use List reference?

Consider:

```
List<String> names = new ArrayList<>();
```

Later you can change implementation:

```
List<String> names = new LinkedList<>();
```

Your variable is still called:

names

This makes code more flexible.

You will understand this much better as you work with interfaces and polymorphism.

58. ArrayList Student Management System

Now let's build the practical project.

Our system will support:

1. Add student
2. Display students
3. Search student
4. Update student
5. Delete student
6. Exit

This is where Collections become useful in a real application.

59. Step 1 — Create Student class

```
class Student {  
  
    int id;  
  
    String name;  
  
    int marks;  
  
    Student(int id, String name, int marks) {  
        this.id = id;  
        this.name = name;  
        this.marks = marks;  
    }  
}
```

Our student object contains:

id

name

marks

Example:

```
Student s1 = new Student(101, "Ravi", 85);
```

Conceptually:

s1

|

+-- id = 101

+-- name = Ravi

+-- marks = 85

60. Why this.id = id?

This is connected to the constructor topic you already learned.

We have:

```
int id;
```

and constructor parameter:

```
int id
```

Both have the same name.

So:

```
this.id = id;
```

means:

```
this.id
```

↓

current object's id

```
id
```

↓

constructor parameter

Therefore:

object id = parameter id

61. Add students to ArrayList

```
ArrayList<Student> students = new ArrayList<>();
```

This means:

Create an ArrayList that can store Student objects.

Then:

```
students.add(new Student(101, "Ravi", 85));
```

```
students.add(new Student(102, "Sita", 92));
```

```
students.add(new Student(103, "John", 78));
```

Now conceptually:

ArrayList

```
|  
+---- Student 101  
  
|  
+---- Student 102  
  
|  
+---- Student 103
```

62. Display students

We can use:

```
for (Student student : students) {
```

```
    System.out.println(student.id);
```

```
    System.out.println(student.name);
```

```
    System.out.println(student.marks);
```

```
}
```

Let's understand:

```
for (Student student : students)
```

means:

Take each Student object from students one by one and temporarily store it in student.

Then:

student.id

gets that student's ID.

63. Better display using toString()

We can override toString().

@Override

```
public String toString() {  
    return "ID: " + id +  
        ", Name: " + name +  
        ", Marks: " + marks;  
}
```

Then:

```
System.out.println(student);
```

will automatically call:

```
student.toString()
```

Output:

ID: 101, Name: Ravi, Marks: 85

64. Search student by ID

Suppose:

101 Ravi

102 Sita

103 John

User enters:

102

We loop:

```
for (Student student : students) {  
  
    if (student.id == id) {  
        System.out.println(student);  
        return;  
    }  
}
```

The program checks:

101 == 102 ✖

102 == 102 ✔

Then prints Sita.

65. Update student

Suppose:

ID: 102

Name: Sita

Marks: 92

User wants:

Marks = 95

Find the object:

```
if (student.id == id) {  
    student.marks = newMarks;  
}
```

Now:

ID: 102

Name: Sita

Marks: 95

Notice:

We don't need to remove the student and add another student.

We directly update the object's field.

66. Delete student

Suppose:

[Student101, Student102, Student103]

Want to delete:

Student102

We can use an iterator safely:

```
Iterator<Student> iterator = students.iterator();
```

```
while (iterator.hasNext()) {
```

```
    Student student = iterator.next();
```

```
    if (student.id == id) {
```

```
        iterator.remove();
```

```
        break;
```

```
    }
```

```
}
```

This is useful when removing elements while iterating.

We'll study Iterator more deeply later in Collections.

67. Complete Student Management System

Here is a beginner-friendly version:

```
import java.util.ArrayList;
```

```
import java.util.Scanner;
```

```
class Student {
```

```
int id;  
String name;  
int marks;
```

```
Student(int id, String name, int marks) {  
    this.id = id;  
    this.name = name;  
    this.marks = marks;  
}
```

```
@Override
```

```
public String toString() {  
    return "ID: " + id +  
        ", Name: " + name +  
        ", Marks: " + marks;  
}  
}
```

```
public class StudentManagement {
```

```
    static ArrayList<Student> students = new ArrayList<>();
```

```
    static Scanner scanner = new Scanner(System.in);
```

```
    public static void addStudent() {
```

```
        System.out.print("Enter student ID: ");  
        int id = scanner.nextInt();
```

```
scanner.nextLine();

System.out.print("Enter student name: ");
String name = scanner.nextLine();

System.out.print("Enter marks: ");
int marks = scanner.nextInt();

Student student = new Student(id, name, marks);

students.add(student);

System.out.println("Student added successfully!");
}

static void displayStudents() {

    if (students.isEmpty()) {
        System.out.println("No students found.");
        return;
    }

    for (Student student : students) {
        System.out.println(student);
    }
}

static void searchStudent() {
```

```
System.out.print("Enter student ID: ");
int id = scanner.nextInt();

for (Student student : students) {

    if (student.id == id) {
        System.out.println(student);
        return;
    }
}

System.out.println("Student not found.");
}

static void updateStudent() {

    System.out.print("Enter student ID: ");
    int id = scanner.nextInt();

    for (Student student : students) {

        if (student.id == id) {

            System.out.print("Enter new marks: ");
            int marks = scanner.nextInt();

            student.marks = marks;

            System.out.println("Student updated successfully!");
```

```

        return;
    }
}

System.out.println("Student not found.");
}

static void deleteStudent() {

    System.out.print("Enter student ID: ");
    int id = scanner.nextInt();

    for (int i = 0; i < students.size(); i++) {

        if (students.get(i).id == id) {

            students.remove(i);

            System.out.println("Student deleted successfully!");
            return;
        }
    }

    System.out.println("Student not found.");
}

public static void main(String[] args) {

    while (true) {

```

```
System.out.println("\n===== Student Management System =====");
```

```
System.out.println("1. Add Student");
```

```
System.out.println("2. Display Students");
```

```
System.out.println("3. Search Student");
```

```
System.out.println("4. Update Student");
```

```
System.out.println("5. Delete Student");
```

```
System.out.println("6. Exit");
```

```
System.out.print("Enter your choice: ");
```

```
int choice = scanner.nextInt();
```

```
switch (choice) {
```

```
    case 1:
```

```
        addStudent();
```

```
        break;
```

```
    case 2:
```

```
        displayStudents();
```

```
        break;
```

```
    case 3:
```

```
        searchStudent();
```

```
        break;
```

```
    case 4:
```

```
        updateStudent();
        break;

    case 5:
        deleteStudent();
        break;

    case 6:
        System.out.println("Program ended.");
        scanner.close();
        return;

    default:
        System.out.println("Invalid choice.");
    }
}
}
```

68. Understand the complete program architecture

Don't just copy the code.

Understand this structure:

Student



Represents one student

ArrayList<Student>



Stores many students

addStudent()



Adds student

displayStudents()



Displays all students

searchStudent()



Finds student

updateStudent()



Changes student information

deleteStudent()



Removes student

main()



Menu / controls program

This is the actual reason Collections are useful.

69. Example execution

Program starts:

===== Student Management System =====

1. Add Student
2. Display Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Enter your choice: 1

User enters:

101

Then:

Enter student name: Ravi

Enter marks: 85

ArrayList becomes:

```
[  
    Student(101, Ravi, 85)  
]
```

Add another:

102

Sita

92

Now:

```
[  
    Student(101, Ravi, 85),  
    Student(102, Sita, 92)  
]
```

Display:

ID: 101, Name: Ravi, Marks: 85

ID: 102, Name: Sita, Marks: 92

70. Very important distinction: Array vs ArrayList

You should be able to answer this in an interview.

Array	ArrayList
Fixed size	Dynamically resizable
Can store primitives directly	Stores objects
length	size()
Can use []	Uses methods such as get()
Less built-in functionality	Many useful methods
Example <code>int[]</code>	Example <code>ArrayList<Integer></code>

Example:

```
int[] numbers = new int[5];
```

versus:

```
ArrayList<Integer> numbers = new ArrayList<>();
```

71. Important List methods to memorize

For your current level, remember these:

`add()`

`add(index, element)`

`get()`

`set()`

`remove()`

`size()`

`contains()`

`isEmpty()`

`clear()`

`indexOf()`

For `LinkedList` additionally:

`addFirst()`

`addLast()`

`getFirst()`

`getLast()`

`removeFirst()`

`removeLast()`

72. Common beginner mistakes

Mistake 1

`ArrayList<int> numbers;`

✗ Wrong.

Use:

`ArrayList<Integer> numbers;`

Mistake 2

`List<String> names = new List<>();`

✗ Wrong.

Use:

`List<String> names = new ArrayList<>();`

Mistake 3

Using:

`names.length`

for `ArrayList`.

✗ Wrong.

Use:

`names.size()`

Mistake 4

Trying:

`names[0]`

✗ Wrong.

Use:

`names.get(0)`

Mistake 5

Confusing `add()` and `set()`.

`add()`

means:

Add an element.

`set()`

means:

Replace an existing element.

73. One very important interview question

Q: Does `ArrayList` store primitive data types?

Strictly speaking:

No. Java collections store objects.

Therefore:

`ArrayList<Integer>`

uses `Integer`, not `int`.

Java automatically converts between primitive and wrapper in many cases through **autoboxing/unboxing**.

Example:

```
ArrayList<Integer> numbers = new ArrayList<>();
```

```
numbers.add(10);
```

Java automatically converts:

`int 10`

↓

`Integer` object

This is called **autoboxing**.

When retrieving:

```
int x = numbers.get(0);
```

Java can convert:

`Integer`

↓

`int`

This is called **unboxing**.

74. Another important interview question

Q: Can List contain duplicate elements?

Yes.

```
List<String> list = new ArrayList<>();
```

```
list.add("A");
```

```
list.add("A");
```

```
list.add("B");
```

Result:

[A, A, B]

75. Is List ordered?

Yes.

List maintains insertion order.

```
list.add("A");
```

```
list.add("B");
```

```
list.add("C");
```

You get:

[A, B, C]

76. Is ArrayList synchronized?

No.

ArrayList

↓

Not synchronized by default

Whereas:

Vector

↓

Synchronized methods

Don't confuse synchronization with "faster" or "better"; it relates to thread safety and concurrent access.

77. ArrayList vs LinkedList — the easiest way to remember

Think like this:

ArrayList

I need fast index access.

Example:

```
list.get(500);
```

ArrayList is well suited.

LinkedList

I need linked/deque-style operations.

Especially:

```
addFirst()
```

```
addLast()
```

```
removeFirst()
```

```
removeLast()
```

78. Your Day 28–30 learning map

Don't try to memorize everything in one sitting.

Day 28 — List + ArrayList

Learn:

Collection Framework



Collection



List



ArrayList

Practice:

```
add()
```

```
get()
```

```
set()
```

```
remove()
```

```
size()
```

```
contains()
```

```
isEmpty()
```

clear()

And:

ArrayList<String>

ArrayList<Integer>

ArrayList<Student>

Day 29 — LinkedList + Vector

Learn:

ArrayList

vs

LinkedList

vs

Vector

Practice:

addFirst()

addLast()

getFirst()

getLast()

removeFirst()

removeLast()

Understand:

Array-backed structure

vs

Linked structure

vs

Legacy synchronized list

Day 30 — Student Management Project

Build:

Student Management System

Features:

Add

Display

Search

Update

Delete

Exit

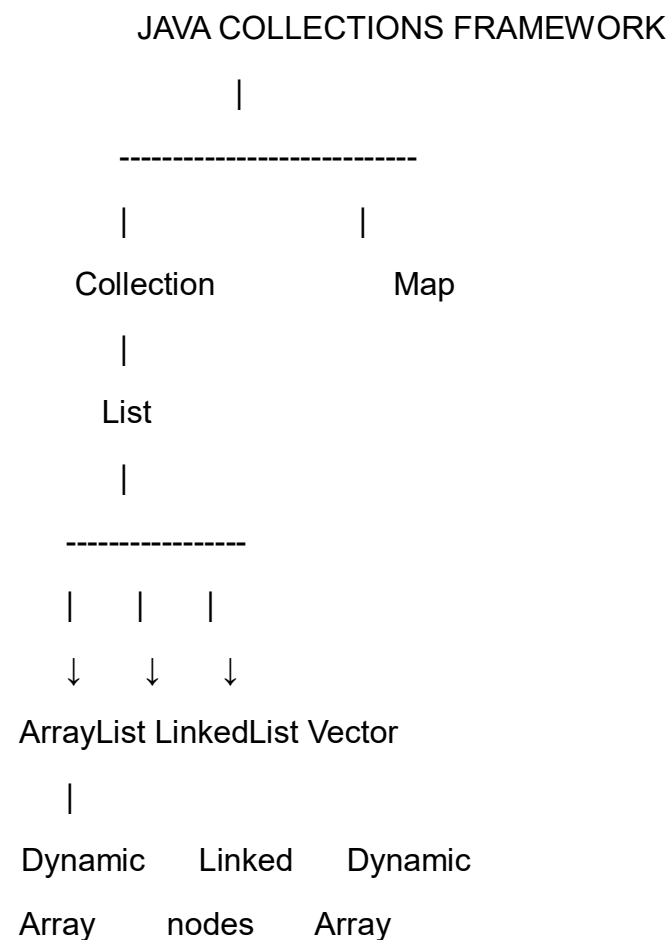
And understand:

`ArrayList<Student>`

very deeply.

79. Final mental picture

If you remember only one diagram from this lesson, remember this:



structure structure structure

List characteristics:

- ☐ Ordered
- ☐ Index-based
- ☐ Allows duplicates
- ☐ Allows null

And:

ArrayList



Best general-purpose List for many cases



Fast index access

LinkedList



Linked nodes



Useful for certain insertion/removal/deque workloads

Vector



Legacy dynamic array



Synchronized

Most important syntax

```
List<String> names = new ArrayList<>();
```

```
names.add("Ravi");
```

```
names.get(0);  
names.set(0, "Sita");  
names.remove(0);  
names.size();  
names.contains("Ravi");  
names.isEmpty();
```

And for your project:

```
ArrayList<Student> students = new ArrayList<>();
```

That single line means:

"Create a dynamically sized list that stores Student objects."

Once this sentence is completely clear, you have understood the core idea behind today's Collections topic.